

# Confidence-Driven Prefetching: Stride Degradation for Hardware Prefetching

Nico Fidalgo

John A. Paulson School of Engineering and Applied Sciences

Harvard University

Cambridge, MA 02138

[nfidalgo@college.harvard.edu](mailto:nfidalgo@college.harvard.edu)

**Abstract**—Hardware prefetching is a widely studied field for reducing memory latency, yet classical stride prefetchers based on the Reference Prediction Table (RPT) [6] design are limited by a case where a single stride break during steady state immediately discards the stored stride, even when the underlying access pattern is otherwise stable. This paper introduces Confidence-Driven Prefetching (CDP), an extension to the per-PC stride prefetcher that replaces the binary STEADY/NO\_PREDICT transition of the original RPT with a saturating confidence counter. CDP stores stride across short bursts of irregularity and only adopts a new stride after confidence has fully decayed. CDP is evaluated against a Chen–Baer RPT baseline and a Feedback-Directed Prefetcher (FDP) [7] on three benchmarks (libquantum, hmmer and dealII) using an Intel Pin cache simulator. CDP reduces effective L1D misses by 37% relative to the Chen–Baer baseline on dealII, ties on libquantum, and slightly outperforms the baseline on hmmer, while matching or beating Global FDP across all three workloads. The hardware cost is one additional bit per RPT entry, a 0.5% increase over the baseline RPT. To contextualize the research, this report also documents an independently evaluated TAGE branch predictor implementation that improves prediction accuracy by 0.78–0.88 percentage points over a two-level adaptive baseline.

## I. INTRODUCTION

The two largest sources of stalls in a modern out-of-order processor are branch mispredictions and cache misses. The hardware designs that mitigate them have been studied for decades, but there remains room for improvement when dealing with practical workloads with diverse access and control-flow behavior. This work investigates both of these problems with a TAGged GEometric history length (TAGE) branch predictor [4] to improve branch prediction accuracy and an exploration of research into extending the classical stride prefetcher of Chen and Baer [6].

The branch prediction component validates a TAGE implementation against the two-level adaptive predictor of Yeh and Patt [3]. TAGE achieves 99.625% prediction accuracy on dealII (a 0.78 percentage point improvement over the two-level baseline) and 99.221% on hmmer (a 0.88 percentage point improvement). On libquantum, all schemes converge near 100% because the benchmark is dominated by easily predictable branches.

The data prefetching component is the principal contribution of this work. The Chen–Baer Reference Prediction Table (RPT)

reduces effective L1D misses by 78% on libquantum and 97% on hmmer when combined with a victim cache. Taking that a step further, an implementation of Feedback-Directed Prefetching (FDP) [7] is used as another baseline for performance comparison. Lastly, Confidence-Driven Prefetching (CDP) is introduced as a novel approach. CDP replaces the STEADY/NO\_PREDICT transition of the RPT finite-state machine (FSM) with a saturating confidence counter that delays stride replacement until the entry has fully decayed. The result is favorable prefetching for patterns that are mostly stable but occasionally perturbed, a category that includes inner loops.

CDP achieves a 37% reduction in effective L1D misses on dealII compared to the Chen–Baer baseline, ties on libquantum where the baseline is already near-optimal, and improves considerably on hmmer. This design adds three bits per RPT entry. A sweep of CDP’s hyperparameters confirmed that nearly all configurations explored beat the Chen–Baer baseline on dealII.

The full source code, evaluation scripts, and raw output files are available at the project repository (see Section ??).

## II. BACKGROUND AND RELATED WORK

### A. Branch Prediction

Branch prediction has progressed from simple bimodal predictors [1] through two-level adaptive schemes [3] to modern tagged geometric history-length predictors. The two-level adaptive predictor uses a per-branch history register table indexed by the program counter and each register selects an entry in a global pattern table of saturating counters. This design is limited by two factors. First, a single fixed history length cannot simultaneously capture short loop-bounded correlations and long data-dependent correlations. Second, aliasing in the history register table allows unrelated branches to corrupt one another’s recorded history.

TAGE [4] addresses both limitations through multiple tagged prediction tables, each indexed by a different length of global branch history. A partial tag in each entry detects aliasing, permitting the predictor to fall back to a table with shorter history when no match is found. Subsequent variants such as TAGE-SC-L incorporate a statistical corrector and a loop predictor. Both have won multiple Branch Prediction Championship competitions and serve as the foundation of branch prediction in commercial processors.

## B. Hardware Data Prefetching

The earliest hardware prefetchers prefetched a fixed offset, typically the next sequential cache line [2]. Stream buffers [5] extend next-line prefetching with a small FIFO queue to absorb cache pollution. Per-instruction stride prefetching [6] associates each load instruction with an RPT entry that tracks the most recent address and the observed stride. A four-state FSM (INIT, TRANSIENT, STEADY, NO\_PREDICT) determines when the prefetcher should issue requests and when it should adapt to a new stride. This RPT design forms the basis of L1 stride prefetchers in many modern processors and is the baseline used for comparison in this research.

Later work has moved in several directions. FDP [7] adjusts a global aggressiveness counter based on prefetch accuracy, prefetch lateness, and prefetch buffer pollution at runtime. The Sandbox prefetcher [8] evaluates candidate offsets in a Bloom filter without committing prefetches to the cache, then commits only the offsets shown to be useful. Michaud’s Best-Offset prefetcher [9] extends this sandbox technique with explicit timeliness tracking through a delay queue.

More recent work has explored multi-stride pattern matching, prefetching based on signature paths, and temporal prefetchers such as Triangel [10] that record and replay correlated address pairs. These approaches achieve higher coverage on irregular access patterns at the cost of substantially larger metadata structures.

A common design assumption across these schemes is that a stride break is treated as evidence that the underlying access pattern has changed. The Chen–Baer FSM transitions immediately to NO\_PREDICT on a stride break and overwrites the stored stride on the next access. FDP modulates aggressiveness globally rather than per-entry. Sandbox prefetchers re-evaluate candidate offsets periodically. None of these schemes maintain per-entry confidence in the stored stride as a quantity that resists overwriting until evidence of sustained change has accumulated. The design proposed in Section III-E addresses this gap.

## III. METHODOLOGY

### A. Simulation Infrastructure

All experiments use Intel Pin as a dynamic binary instrumentation framework. A Pin tool intercepts every instruction fetch, load, and store, and routes the resulting addresses through a software model of the cache hierarchy. The hierarchy comprises a 32 KB direct-mapped L1D, a 32 KB 4-way L1I, and a 256 KB 8-way unified L2, with all caches using 64-byte lines. Optional structures include a fully-associative victim cache and a fully-associative prefetch buffer parallel to the L1D. Effective L1D misses are defined as raw L1D misses minus victim cache hits minus prefetch misses.

The three SPEC benchmarks that are used in this work are libquantum (a quantum simulation kernel dominated by a single regular access stream), hmmer (a protein database search with mixed loads), and dealII (a finite element computation with multiple concurrent stride patterns). Each is compiled with O3 optimization and executed under Pin for 200 million

instructions only which is a limitation of the research although sufficient to get to steady-state behavior of all prefetchers and predictors evaluated to converge.

### B. TAGE Branch Predictor

The TAGE implementation supports five predictor modes selectable at runtime: a 1-bit branch prediction buffer (BPB), a 2-bit BPB using saturating A2 counters [3], a two-level adaptive predictor with a hash history register table and global pattern table, gshare, and TAGE.

The TAGE predictor has a bimodal base predictor  $T_0$  indexed by PC and  $N$  tagged components  $T_1 \dots T_N$  indexed by hashes that use the PC and a length of global branch history. The history lengths are  $\{5, 13, 32, 80, 200\}$  bits and configurations sweep  $N \in \{2, 4, 6\}$  and tagged table sizes of  $\{512, 2048, 8192\}$  entries. Each tagged entry stores a 3-bit signed prediction counter, an 11-bit partial tag, and a 2-bit useful counter.

On a prediction request, all tagged components are probed in parallel. The provider is the longest-history component with a tag match and the altpred is the next longest. On a misprediction, a new entry is allocated in a table with a longer history and if all candidate slots have nonzero useful counters, those counters are decremented to make space at the next allocation attempt. Useful counters are periodically right-shifted every  $2^{18}$  branches to prevent saturation from blocking allocation indefinitely as a backstop.

### C. Stride Prefetcher Baseline

The Chen–Baer classical implementation has a 64-entry RPT with the four-state FSM from the paper. The state transitions are INIT to TRANSIENT on the first stride observation, TRANSIENT to STEADY on stride confirmation, STEADY to NO\_PREDICT on stride mismatch, and NO\_PREDICT to TRANSIENT on stride re-confirmation. Prefetches are issued only when an entry is in STEADY. The prefetch buffer holds 16 entries and demand hits in the buffer are counted as serviced misses and contribute to the effective miss reduction as mentioned previously.

### D. Feedback-Directed Prefetching

The FDP implementation tracks prefetch accuracy (prefetch buffer hits divided by prefetches issued), pollution (prefetch buffer evictions of unused lines divided by inserts), and a lateness proxy (demand accesses on lines still in flight to L2) at runtime. A global aggressiveness counter, updated every 100,000 instructions, maps to one of four prefetch degrees in  $\{1, 2, 4, 8\}$ . The counter increments when accuracy exceeds 75% and pollution is below 25%, and decrements when either accuracy drops below 40% or pollution exceeds 25%.

Note, when conducting this research an initial attempt to implement FDP added a per-entry aggressiveness counter to the RPT, motivated by the observation that load instructions within a single benchmark have heterogeneous access regularity. Empirically, this design did not outperform the global FDP scheme. This was the result of hash collisions in the RPT

which caused regular and irregular loads to share the same slot. This negative result motivated the principal design described next which avoids per-PC bookkeeping altogether.

### E. Confidence-Driven Prefetching

CDP replaces the four-state FSM with a saturating confidence counter  $c \in [0, c_{\max}]$  per RPT entry. The default  $c_{\max} = 7$  requires three bits of storage per entry. On each demand access that matches an existing RPT entry, the entry's confidence is updated according to whether the observed stride matches the stored stride:

$$c \leftarrow \begin{cases} \min(c + 1, c_{\max}) & \text{if obs = stride} \\ \max(c - 1, 0) & \text{otherwise} \end{cases} \quad (1)$$

The key difference from Chen–Baer is the stored stride. In the classical Chen–Baer design, a stride mismatch in STEADY transitions to NO\_PREDICT, and the next mismatched access in NO\_PREDICT overwrites the stored stride. CDP defers this overwrite until the confidence has fully decayed to zero:

$$\text{stride} \leftarrow \begin{cases} \text{obs} & \text{if obs} \neq \text{stride and } c = 0 \\ \text{stride} & \text{otherwise} \end{cases} \quad (2)$$

This invariant is the core of the design. A single outlier access decrements confidence by one but leaves the stored stride intact. Only after the entry's confidence has fully drained, indicating sustained inconsistency rather than transient noise, does the entry adopt a new stride. Stable patterns persist across short bursts of irregularity, while genuinely irregular loads drain to zero confidence and are filtered out of the prefetch issue path.

A configurable threshold  $t_{pf}$  governs when prefetches begin. When  $c < t_{pf}$  the entry issues no prefetches, otherwise it issues a fixed degree prefetch on each access. Two additional thresholds  $t_{d2}$  and  $t_{d4}$  allow confidence to scale degree, but empirical evaluation revealed that degree-2 and degree-4 prefetching saturate the small prefetch buffer and produce more pollution than benefit on the benchmarks studied. The default configuration uses  $t_{pf} = 1$  and caps the degree at 1, so CDP issues one prefetch per access once the entry's confidence has reached the threshold.

A guard on the first access handles the boundary case in which an RPT entry has just been allocated and its stored stride is zero. On the first nonzero observed stride, the entry records the value and returns without updating confidence or issuing a prefetch. This prevents an incorrect zero stride match from inflating confidence on a newly-allocated entry.

The hardware cost of CDP relative to the Chen–Baer baseline is one additional bit per RPT entry, since the 3-bit confidence counter replaces the 2-bit FSM state. For a 64-entry RPT, this is 64 additional bits, or 0.5% of the baseline RPT cost of 12.4 Kbits. The state machine logic is replaced by a saturating counter and a comparator, both of which are already present in any RPT implementation that tracks transient state.

TABLE I: Best branch prediction accuracy (%) by scheme.

| Predictor               | dealII | hmmer  | libquantum |
|-------------------------|--------|--------|------------|
| 1-bit BPB               | 89.60  | 96.50  | 99.990     |
| 2-bit BPB (A2)          | 94.48  | 97.50  | 99.992     |
| 2-level adaptive        | 98.84  | 98.34  | 99.990     |
| TAGE ( $n=6, ts=8192$ ) | 99.625 | 99.221 | 99.992     |

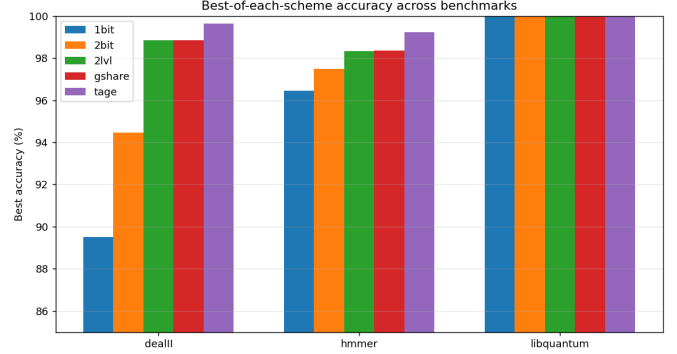


Fig. 1: Best prediction accuracy by scheme across benchmarks. TAGE achieves the highest accuracy on all three workloads.

## IV. RESULTS

### A. TAGE Branch Predictor Accuracy

Table I reports the best prediction accuracy achieved by each scheme on each benchmark using a 10 million branch limit. TAGE achieves the highest accuracy on every benchmark.

The improvement of TAGE over the two-level baseline is most pronounced on dealII (0.78 percentage points) and hmmer (0.88 percentage points). On libquantum, all schemes above 2-bit converge near 100% because the benchmark's branches are dominated by highly predictable behavior.

Figure 1 summarizes the best accuracy attained by each scheme. Figure 2 shows the design space exploration of prediction accuracy as a function of total hardware budget in bits and sweeping TAGE configurations  $(n, ts) \in \{2, 4, 6\}, \{512, 2048, 8192\}$ .

The hardware budget for the best TAGE configuration is approximately 800 Kbits, comparable to branch predictors in commercial processors. TAGE crosses the two-level baseline at budgets as low as 32 Kbits, demonstrating favorable efficiency even with small hardware budgets.

### B. Stride Prefetcher Baseline

Table II shows effective L1D misses for the Chen–Baer stride prefetcher implementation against the direct-mapped baseline, the 8-way associative baseline, and victim cache configurations.

On libquantum, where misses are predominantly capacity misses caused by streaming access to a large quantum state vector, the victim cache reduces misses by 13% while the stride prefetcher reduces them by 75%. On hmmer, the combined stride and victim cache configuration reduces misses by 97% which show that these two designs address the same cache miss types. On dealII, where the working set greatly exceeds the

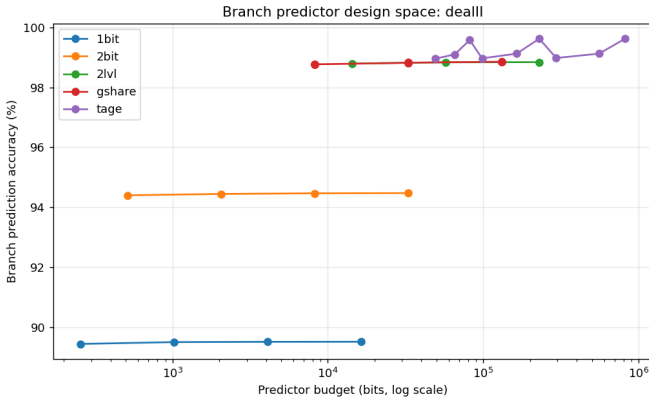


Fig. 2: Branch predictor design space on dealII: accuracy versus hardware budget on a logarithmic scale. TAGE configurations monotonically improve with increased budget, surpassing the two-level baseline at budgets as low as 32 Kbits.

TABLE II: Effective L1D misses for the Chen–Baer baseline and victim cache configurations on a direct-mapped L1D.

| Configuration              | libquantum | hmmr       | dealII     |
|----------------------------|------------|------------|------------|
| Direct-mapped, no prefetch | 2,089,886  | 15,080,134 | 10,815,111 |
| 8-way, no prefetch         | 1,805,256  | 1,922,707  | 1,853,748  |
| 4-entry victim cache       | 1,810,456  | 5,351,646  | 8,954,916  |
| Stride RPT (64 entries)    | 512,255    | 4,198,251  | 487,029    |
| Stride RPT + 4-entry VC    | 462,130    | 517,561    | 510,685    |

cache capacity and misses are dominated by capacity misses, the stride prefetcher reduces misses by 95% while the victim cache alone achieves only a 17% reduction.

### C. Confidence-Driven Prefetching

Table III compares CDP against the Chen–Baer baseline and Global FDP across all three benchmarks at 200 million instructions.

CDP achieves a 37.2% reduction in effective L1D misses on dealII, a 2.7% reduction on hmmr, and nearly ties on libquantum as seen in Figure 3. A tie on libquantum is expected as the benchmark’s access pattern is captured by a single stable RPT entry that quickly saturates to maximum confidence under any reasonable threshold setting, leaving CDP and the Chen–Baer baseline functionally equivalent for that workload.

The dealII improvement is the key contribution of this work. dealII’s finite element computation traverses multiple data structures concurrently, each with its own stride pattern. These patterns are largely stable but include occasional noise from boundary handling, refinement transitions, and conditional code paths that introduce one-off stride breaks. The Chen–Baer FSM treats each perturbation as a signal that the entry’s stored stride should be replaced, even when the underlying pattern is otherwise stable. CDP defers this replacement until confidence has fully decayed, preserving prefetching opportunities. Figures 4 and 5 show this comparison across benchmarks.

TABLE III: Effective L1D misses comparing the Chen–Baer baseline, Global FDP, and CDP. Best result per benchmark in bold.

| Configuration                 | libquantum     | hmmr             | dealII         |
|-------------------------------|----------------|------------------|----------------|
| No prefetch                   | 2,090,375      | 15,173,803       | 9,088,432      |
| Chen–Baer (fixed)             | <b>510,417</b> | 4,048,589        | 563,507        |
| Global FDP                    | 510,685        | 4,010,934        | 567,663        |
| CDP                           | 510,488        | <b>3,943,843</b> | <b>354,033</b> |
| CDP improvement vs. Chen–Baer | 0.0%           | 2.7%             | 37.2%          |

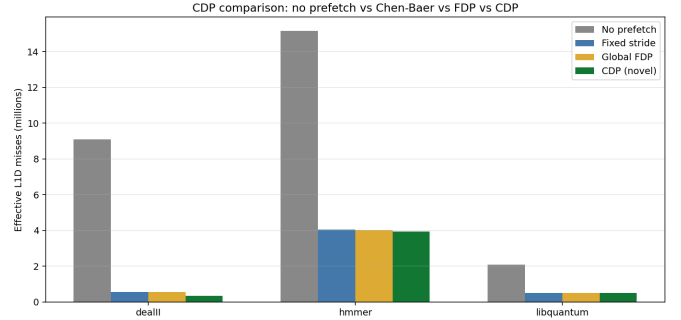


Fig. 3: Effective L1D miss comparison across all three benchmarks. CDP outperforms the Chen–Baer baseline most dramatically on dealII, where multiple concurrent stride patterns benefit from the protective stride retention mechanism.

### D. Threshold Sensitivity

The threshold sweep evaluates 21 distinct  $(t_{pf}, t_{d2}, t_{d4})$  configurations on each benchmark. Figure 7 shows the dealII results.

The dealII results show that CDP’s improvement is robust to threshold selection as only one configuration in the sweep underperforms the Chen–Baer baseline, and the majority cluster within 5% of the optimal configuration. This robustness is a desirable property because it suggests the design is not finely tuned to a specific workload but reflects a general improvement in stride prefetcher behavior.

The libquantum and hmmr threshold sweeps are flat in that all configurations produce nearly identical results, consistent with the observation that on these workloads the population of high-confidence entries is small and stable, leaving little room for threshold-driven variation.

### E. Confidence Distribution and the Bimodal Steady State

The confidence histograms in Figures 8, 9, and 10 reveal the distribution of final confidence values is bimodal across all three benchmarks, with entries clustered at either  $c = 0$  or  $c = c_{\max} = 7$  and few entries at intermediate values.

This pattern is due to the execution of the benchmarks’ inner loops millions of times within the 200 million instruction window. Any entry that achieves  $c \geq t_{pf}$  rapidly saturates to  $c_{\max}$  as matching strides accumulate and conversely any entry that fails to maintain a stable stride decays to zero. Intermediate confidence values exist only during pattern establishment or

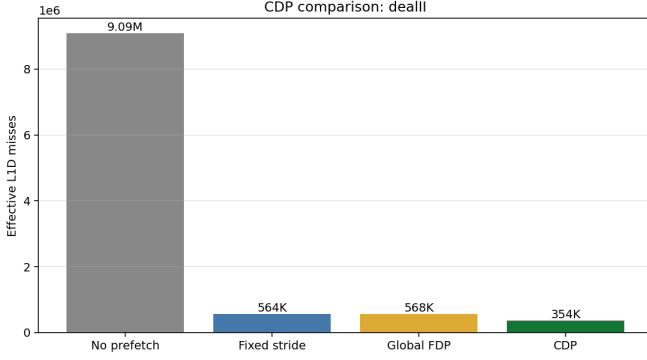


Fig. 4: CDP comparison on dealII. CDP reduces effective L1D misses from 564K to 354K, a 37% improvement over the Chen–Baer baseline. Global FDP performs essentially identically to the baseline, indicating that adaptive global aggressiveness alone is insufficient to capture the benefit available on this workload.

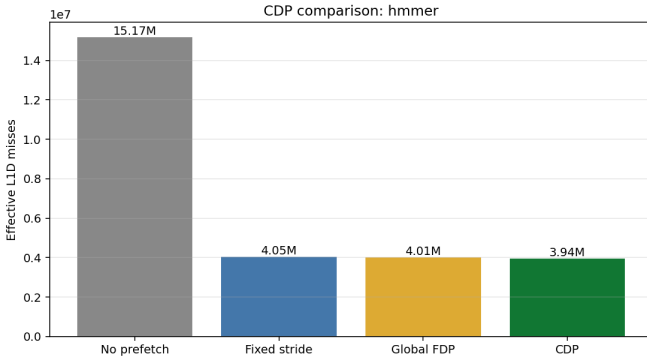


Fig. 5: CDP comparison on hmmer. The improvement is modest at 2.7%, reflecting hmmer’s mix of regular and irregular loads. The discriminative effect of CDP’s confidence counter is visible in the confidence distribution (Figure 8) where roughly half of allocated entries saturate at maximum confidence, while the other half remain at zero.

during transient breaks, and are unlikely to appear in the final snapshot.

Note, the bimodal distribution should not be read as evidence that the intermediate confidence range is unused. The intermediate states are essential for correct transitions as they delay the loss of a stride during transient breaks and the acquisition of a new stride until evidence accumulates. The bimodal final state is the equilibrium toward which the dynamics converge, not a sign of underutilization and are therefore desirable.

#### F. Hardware Cost Analysis

The Chen–Baer RPT entry is comprised of a 64-bit PC tag, a 64-bit last address, a 64-bit stride, and 2 bits of FSM state, for a total of 194 bits per entry. CDP replaces the 2-bit FSM state with a 3-bit confidence counter, increasing the entry size

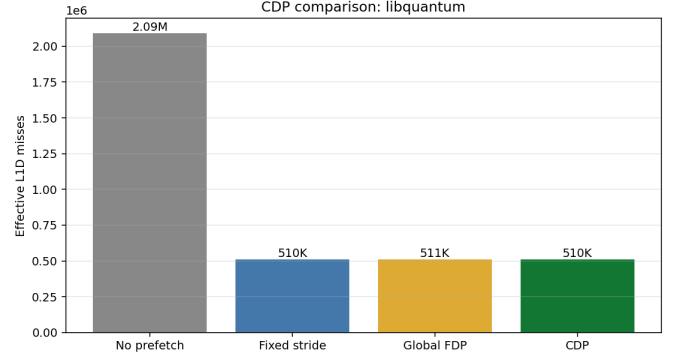


Fig. 6: CDP comparison on libquantum. All three prefetcher configurations achieve effectively identical results because the benchmark’s dominant access pattern is captured by a single high-confidence entry under any of the three schemes.

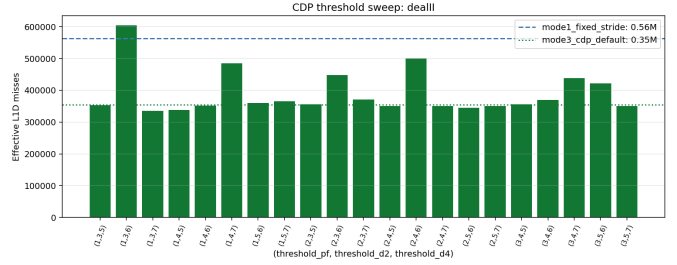


Fig. 7: Threshold sensitivity on dealII. The dashed line marks the Chen–Baer baseline (564K effective misses). The dotted line marks the CDP default configuration. Nearly every threshold combination explored beats the baseline; the worst-performing configurations correspond to extreme combinations such as  $t_{pf} = 1, t_{d2} = 3, t_{d4} = 6$ , which trigger frequent transitions between degree levels. The default (2, 4, 8) configuration sits near the bottom of the achievable range.

by 1 bit to 195 bits. At 64 entries the total RPT cost increases from 12.416 Kbits to 12.480 Kbits, an increase of 0.5%.

For comparison, Global FDP requires 2 bits of global aggressiveness state, three counters of approximately 32 bits each for tracking window-aggregated metrics, and the same 64-entry RPT, totaling 12.514 Kbits. CDP and FDP are therefore comparable in cost, but CDP delivers larger improvements.

#### V. DISCUSSION

The key finding of this work is that the abrupt stride replacement in the Chen–Baer FSM is a meaningful source of lost prefetching opportunity on workloads with mostly stable access patterns interrupted by occasional perturbations. The extent of the loss varies dramatically across workloads. On libquantum for instance, where the dominant access pattern is genuinely stable, no opportunity is lost. On the other hand, with dealII, where multiple concurrent stride patterns each suffer occasional perturbations, the opportunity is substantial.

A natural concern with the proposed design is that protecting the stored stride might delay correct adaptation when an

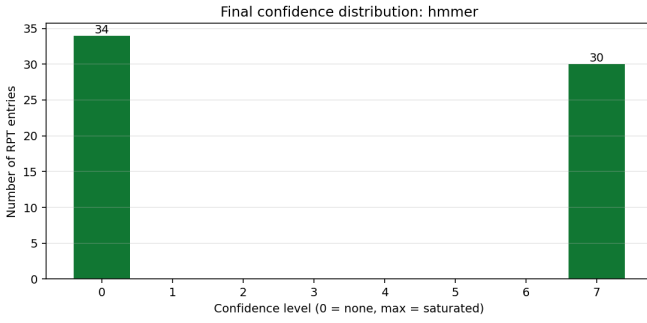


Fig. 8: Final confidence distribution at the end of execution for hmmer. 30 entries have saturated to  $c_{\max} = 7$  (loads with stable strides), and 34 entries remain at  $c = 0$  (loads with irregular access patterns). Intermediate values are transient, occurring only briefly during pattern establishment or stride change.

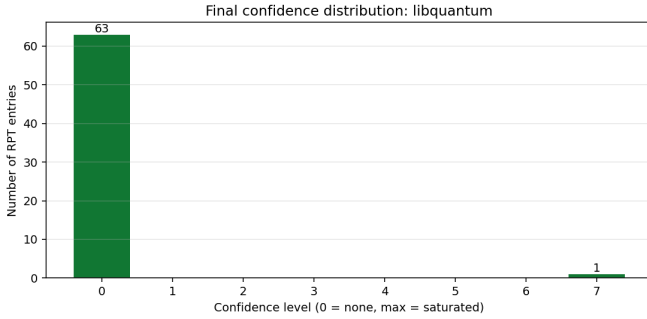


Fig. 9: Final confidence distribution on libquantum. The single saturated entry exemplifies the dominant access pattern of libquantum’s quantum simulation loop.

access pattern genuinely shifts. The threshold sweep results indicate that this concern is not observed practically as the workloads evaluated do not exhibit sustained pattern shifts that confidence-driven stride retention would delay as seen in the very similar performance in the sweep of threshold combinations in Figure 7.

Although, there could be an alternative interpretation of the dealII improvement. The Chen–Baer FSM uses two bits of state and adapts within two accesses while CDP’s three-bit confidence counter requires multiple matching strides to reach saturation. One could argue that the dealII benefit comes from the slower decay alone, not from the preservation of the stored stride during decay. The implementation, however, preserves the stored stride only when  $c > 0$ , which is also the when CDP issues prefetches. The benefit therefore necessarily depends on the preserved stride as any prefetch issued when  $0 < c < c_{\max}$  uses the stored value, which the Chen–Baer FSM would have already overwritten.

The bimodal confidence distribution discussed in Section IV-C suggests that further work could explore adaptive thresholds. If steady-state behavior collapses to min or max confidence quickly, the threshold parameters  $t_{pf}$ ,  $t_{d2}$ ,  $t_{d4}$

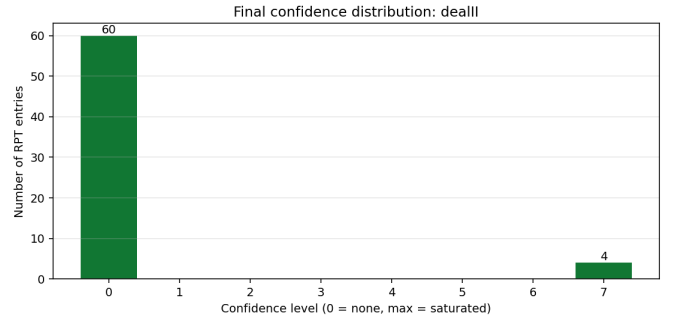


Fig. 10: Final confidence distribution on dealII. The four saturated entries demonstrate the inner loops of dealII’s finite element kernel that traverse data structures with stable strides.

exert influence only during the transitions, and a smaller confidence range ( $c_{\max} = 3$  rather than 7, requiring only 2 bits) might suffice, reducing hardware cost. The threshold sweep at  $c_{\max} = 3$  was evaluated and produced results within 3% of the  $c_{\max} = 7$  default on all benchmarks, suggesting that two bits of confidence may be sufficient.

One limitation of this work is that the simulator does not model precise prefetch timing. A real processor would observe different effective miss rates depending on whether prefetches arrive before, during, or after demand requests. Additionally, the 200 million instruction window captures steady-state behavior as results converge within this window, but over longer instruction windows phase transitions could be captured. Also, the benchmarks are restricted to three workloads and a broader evaluation would test whether CDP’s benefit extends beyond the class of workloads represented here.

## VI. CONCLUSION

This work makes two contributions. First, it validates a TAGE branch predictor implementation that improves prediction accuracy by 0.78 percentage points on dealII and 0.88 percentage points on hmmer over a two-level adaptive baseline and demonstrated the hardware cost versus accuracy tradeoff. Second, and more substantively, it introduces Confidence-Driven Prefetching (CDP), an extension to the Chen–Baer Reference Prediction Table (RPT) that replaces binary stride retention with stride degradation through a saturating confidence counter. The core insight is that an isolated stride break should be treated as evidence of noise, not as evidence of pattern change, until confidence has fully decayed.

CDP achieves a 37% reduction in effective L1D misses on dealII relative to the Chen–Baer baseline, ties on libquantum where the baseline is already near-optimal, and slightly improves on hmmer. It outperforms Global Feedback-Directed Prefetching (FDP) on all three benchmarks. The CDP design increases total RPT hardware cost by 0.5%. CDP’s performance is robust to threshold selection, suggesting that the improvement generalizes to stride prefetcher behavior rather than tuning to a specific workload.



The bimodal confidence distribution suggests that smaller confidence counters (two bits rather than three) might suffice and future work could explore this. The interaction between confidence-based stride retention and the larger family of offset and signature-based prefetchers, particularly Best-Offset, could also be explored. The same insight may apply to the offset selection mechanism in those prefetchers. Finally, evaluation across a broader benchmark suite would strengthen the generality of the results.

## REFERENCES

- [1] J. E. Smith, “A study of branch prediction strategies,” in *Proc. Int. Symp. Computer Architecture (ISCA)*, 1981.
- [2] A. J. Smith, “Cache memories,” *ACM Computing Surveys*, vol. 14, no. 3, 1982.
- [3] T.-Y. Yeh and Y. N. Patt, “Two-level adaptive training branch prediction,” in *Proc. Int. Symp. Microarchitecture (MICRO)*, 1991.
- [4] A. Seznec and P. Michaud, “A case for (partially) tagged geometric history length branch prediction,” *Journal of Instruction-Level Parallelism*, 2006.
- [5] N. P. Jouppi, “Improving direct-mapped cache performance by the addition of a small fully-associative cache and prefetch buffers,” in *Proc. Int. Symp. Computer Architecture (ISCA)*, 1990.
- [6] T.-F. Chen and J.-L. Baer, “Effective hardware-based data prefetching for high-performance processors,” *IEEE Transactions on Computers*, vol. 44, no. 5, pp. 609–623, 1995.
- [7] S. Srinath, O. Mutlu, H. Kim, and Y. N. Patt, “Feedback directed prefetching: Improving the performance and bandwidth efficiency of hardware prefetchers,” in *Proc. Int. Symp. High-Performance Computer Architecture (HPCA)*, 2007.
- [8] S. H. Pugsley et al., “Sandbox prefetching: Safe run-time evaluation of aggressive prefetchers,” in *Proc. Int. Symp. High-Performance Computer Architecture (HPCA)*, 2014.
- [9] P. Michaud, “Best-offset hardware prefetching,” in *Proc. Int. Symp. High-Performance Computer Architecture (HPCA)*, 2016.
- [10] S. Ainsworth and L. Mukhanov, “Triangel: A high-performance, accurate, timely on-chip temporal prefetcher,” in *Proc. Int. Symp. Computer Architecture (ISCA)*, 2024.